

Test Suite Minimization with Mutation Testing-Based Many-Objective Evolutionary Optimization

1st Zheng Wei,
School of Software and
Microelectronics
Northwestern Polytechnical
University
Xi'an, China
wzheng@nwpu.edu.cn

2nd Wu Xiaoxue
School of Automation
Northwestern Polytechnical
University
Xi'an, China
wuxiaoxue00@gmail.com

3rd Yang Xibing
School of Software and
Microelectronics
Northwestern Polytechnical
University
Xi'an, China
yangxibing@nwpu.edu.cn

4th Cao Shichao
School of Software and
Microelectronics
Northwestern Polytechnical
University
Xi'an, China
caoshichao@nwpu.edu.cn

5th Liu Wenxin
School of Software and
Microelectronics
Northwestern Polytechnical
University
Xi'an, China
liuwenxin@nwpu.edu.cn

6th Lin Jun
School of Software and
Microelectronics
Northwestern Polytechnical
University
Xi'an, China
linjun@nwpu.edu.cn

Abstract—

Context : As software evolves, the test suite tends to grow, regression testing has become prohibitively expensive. Test suite minimization is one of the most important approaches for reducing test cost. The process of test suite minimization is a trade-off between cost and other value criteria and is appropriate to be described as a many-objective optimization problem. **Objective:** To identify the most efficient test suite for reducing the redundant degree of test data and improving test efficiency without decreasing the defect detection ability of test data. **Method:** We introduce a mutation testing-based many-objective optimization approach, which gives higher priority to the fault detection ability and takes mutation score as a major objective, together with cost and three standard code coverage criteria for test suite minimization. Six classical evolutionary many-objective optimization algorithms are applied to identify efficient test suite. Three programs from the SIR repository and one larger program, space are applied for empirical study and effectiveness evaluation. **Results:** On the one hand, in three SIR programs experiments NSGA-II with tuning was the most effective technique. However, MOEA/D-PBI outperformed NSGA-II on the larger program (Space). On the other hand, the test cost of the optimal test suite which obtained by the many-objective optimization approach with mutation score is much lower than the one without it in *tcas*. **Conclusions:** The experimental results prove that the many-objective optimization model with the guidance of mutation score is indeed effective in reducing the test suite redundancy.

Keywords—Test suite minimization, many-objective optimization, Mutation score, Cost

I. INTRODUCTION

A. Motivation and Literature Review

With the increasing of software system scale, complexity, and frequency of release, executing all test cases after each release is

prohibitively expensive and time-consuming[1]. To maximize the effectiveness of regression testing, researchers have proposed various approaches in the past[2~8]. Test suite minimization seeks to eliminate obsolete test cases by seeking trade-offs between test cost and fault detection ability, and identify the subset of the original test suite that is deemed to be redundant. Test suite minimization can be formally described as: *given a test suite* $T = \{t_1, t_2, \dots, t_n\}$ *and a set of test requirements* $R = \{r_1, r_2, \dots, r_m\}$ *the problem of test suite minimization is to find the smallest* T' *such that* $T' \subset T$, $\forall r \in R (T' \text{ satisfies } r)$.

The test requirements $R = \{r_1, r_2, \dots, r_m\}$ always involve multiple criteria, such as less execution time, high coverage (such as statement/branch) rate, and high fault detection ability, which may be conflicting and incommensurable. To trade off the different features, this problem can be treated as a many-objective optimization problem. Yoo and Harman [10] appear to be the first researchers to apply a many objective optimization approach to the test suite minimization problem. They implemented a many-objective approach, applying a greedy algorithm and two versions of non-dominated sorting genetic algorithm II(NSGA-II). After that, Zheng and Hierons [11] further developed the many-objective agenda pioneered by Yoo and Harman by adapting a decomposition-based many-objective evolutionary algorithm(MOEA/D).

A central question in the study of test adequacy criteria is that how they relate to fault detecting ability. Existing works are important steps that led to valuable results and insights. However, much of them focus on test suite minimization in the context of cost and code coverage based testing. Though statement coverage, branch coverage, and MC/DC coverage are standard well-known coverage criteria, as a matter of fact, in many cases the tests that

have been written are sufficient to generate 100% branch coverage, but the code can be mutated such that its functionality

is changed but all tests still pass. Some common types of missing test across multiple code bases, as shown in Table 1[12].

TABLE I. MOTIVATION CASES: 100% CODE COVERAGE TESTS MISSING FAULTS

NO.	PUT	TC
Case 1	<pre> public static String foo(int i) { if (i >= 0) { return "foo"; } else { return "bar"; } } </pre>	<pre> @Test public void shouldReturnFooWith1() { assertEquals("foo", foo(5)); } @Test public void shouldReturnBarWithMinus1() { assertEquals("bar", foo(-1)); } </pre>
Case 2	<pre> public static String foo(boolean b) { if (b) { performVitalyImportantBusinessFunction(); return "OK"; } return "FAIL"; } </pre>	<pre> @Test public void shouldFailWhenGivenFalse() { assertEquals("FAIL", foo(false)); } @Test public void shouldBeOkWhenGivenTrue() { assertEquals("OK", foo(true)); } </pre>
Case 3	<pre> public static String foo(Collaborator c, boolean b) { if (b) { return c.performAction(); } return "FOO"; } </pre>	<pre> @Test public void shouldPerformActionWhenGivenTrue() { foo(mockCollaborator,true); verify(mockCollaborator).performAction(); } @Test public void shouldNotPerformActionWhenGivenFalse() { foo(mockCollaborator,false); verify(never(),mockCollaborator).performAction(); } </pre>

In Table 1, for Case 1, though the test suite including two unite test cases for the program under test(PUT) could satisfy 100 percentage code coverage (statement coverage, branch coverage and MC/DC), it indeed missed the boundary value and the behavior when i equals 0 is never tested, which means if the programmer coded ($i > 0$) instead of ($i \geq 0$) mistakenly, the defect could not be detected by the current test suite. For Case 2, similarly, the two test cases which could satisfy 100 percentage code coverage failed to check whether the function *performVitalyImportantBusinessFunction()* is executed or not. And for Case 3, the return value of the function is never checked by the two test cases which also satisfy 100 percentage code coverage. This list of three cases is just some common types of missing test, but is by no means exhaustive.

We hope the test suites could find more faults and support coverage criteria as a proxy for effectiveness. Fault-based testing techniques such as mutation testing is often advocated to overcome limitations of coverage-based testing. Mutation testing, originally proposed by DeMillo [13] in 1978, has been studied by numerous researchers. Walsh [14] found empirically that mutation testing is more powerful than statement and branch coverage. Frankl et al. [15] and Offutt et al. [16] affirm that mutation testing is more successful in finding faults than data flow based testing. Andrews et al. show in a recent empirical study reported in [17] that mutation analysis is potentially useful to assess and compare test suites and criteria in terms of their cost effectiveness.

B. Our Contributions

Be inspired by above works, we developed, and present in this paper, a mutation testing-based many objective optimization approach for test suite minimization. The major contributions of this paper are as follows.

- The first time combining mutation testing and many objective optimization techniques for test suite minimization.
- Applying the approach in six classical evolutionary algorithms.
- Experimentally evaluating the effectiveness of the proposed approach and six evolutionary algorithms in both small program set SIR and a large scale program Space.

C. Organization

The rest of this paper is structured as follows. Section 2 introduce main concepts and mechanism of related theories. Section 3 explains the proposed mutation testing-based many-objective optimization approach to address the test suite minimization problem. Section 4 describes research questions and explains the experimental design. Section 5 analyzes the results of the experiments by answering the research questions. Section 6 draws conclusion and discusses possible lines of future work.

II. PRELIMINARIES

A. Mutation Testing

Let m be a syntactically correct program obtained by making a syntactic change in P ; Let r be a rule according to which P is changed. A test set T can be evaluated against the mutation criteria by executing each mutant against every test case of T . Here we present definitions of major related terms.

Definition 1 (Mutation Operator): A transformation rule r , according to which P is changed to M is called mutation operator.

There could potentially be an infinite number of mutant operators. To keep mutation testing practical, the set of mutant

operators is kept small and consists of simple mutant operators which is applicable and suitable for the target program.

For instance, Table 2 shows the mutant m , generated by changing the operator ($\geq$) of the original program P , into the ($>$), thereby producing the mutant m .

TABLE II. EXPERIMENTAL SUBJECTS

Program P	Mutant m
...	...
if ($i \geq 0$) { return "foo" }	if ($i > 0$) { return "foo" }
...	...

Definition 2 (Mutant). A mutant m is a copy of the original system P with a small syntactic change that can suppose a fault.

The application of a set of mutant operators $R(r \in R)$ to P results in a set of mutants M . Each mutant $m(m \in M)$ holds at least one artificial change.

All test cases of test suite T are executed against the original program P to check its correctness. If P is incorrect, it has to be fixed before running other mutants, otherwise each mutant $m(m \in M)$ will then be run against this test set T . If the result of running m is different from the result of running P for any test case t in T , then the mutant m is said to be ‘killed’, otherwise it is said to have ‘survived’. After all test cases have been executed, there may still be a few ‘surviving’ mutants. To improve the test set T , the program tester can provide additional test inputs to kill these surviving mutants. However, there are some mutants that can never be killed, because they always produce the same output as the original program. These mutants are called Equivalent Mutants.

Definition 3 (Equivalent Mutant). A mutant which is syntactically different but functionally equivalent to the original program.

Definition 4 (Mutation Score). Mutation score is the ratio of the number of killed mutants over the total number of non-equivalent mutants, which includes the not covered mutants too. It can be defined as Eq. (1).

$$MS(P, T) = \frac{K}{M-E} \quad (1)$$

Where P indicates program under test, T indicates test suite of P ; and K is the number of killed mutants, M is the number of generated mutants and E is the number of equivalent mutants. The goal of mutation analysis is to raise the mutation score to 1, indicating the test set T is sufficient to detect all the faults denoted by the mutants. Mutation score is the traditional adequacy measurement of the test suite in mutation testing.

B. Cost

The cost of regression testing is a cross-cutting concern for all classes of regression testing techniques. Studies of cost-aware regression testing can be classified into two categories. First, there is work that aims to incorporate the cost of testing directly into regression testing techniques at technical level; for example, cost-aware test case prioritization or minimization techniques that provide the tester with a series of alternative subsets of the original test suites that can be executed in different amounts of time. Second, there are empirical evaluations of regression testing techniques that consider whether the application of these techniques is indeed cost-effective in the wider context of the overall software lifecycle.

In this paper, when we talk about cost, we only consider the time cost spent on executing the test cases. We define it as definition 5.

Definition 5 (Testing Cost): The time spent on executing the test (can be a test case or a test set).

C. Code Coverage Criteria

Code coverage is a measure used to describe the degree to which the source code of a program is executed when a particular test suites runs. The definitions of major terms are as following.

Definition 6 (Statement Coverage). The percentage of program statements covered in a testing.

Definition 7 (Branch Coverage). The percentage of program branches executed in testing.

Definition 8 (Condition). A Boolean expression containing no Boolean operators.

Definition 9 (Decision). A Boolean expression composed of conditions and zero or more Boolean operators. A decision without a Boolean operator is a condition. If a condition appears more than once in a decision, each occurrence is a distinct condition.

Definition 9 (Decision Coverage). Every point of entry and exit in the program has been invoked at least once and every decision in the program has taken on all possible outcomes at least once.

Definition 10 (MC/DC – Modified Condition/Decision Coverage). Every point of entry and exit in the program has been invoked at least once, every condition in a decision in the program has taken all possible outcomes at least once, every decision in the program has taken all possible outcomes at least once, and each condition in a decision has been shown to independently affect.

III. THE PROPOSED APPROACH

This section describes the detail of the presented test suite minimization approach with many-objective optimization approaches which based on mutation analysis.

A. Selected Objectives

In this paper, we chose 5 factors as test suite minimization objectives, they are mutation score, cost, statement coverage, branch coverage and MC/DC.

Let us suppose that we have test suite $T = \{t_1, t_2, \dots, t_n\}$ for program p and for a test case t_i , $1 \leq i \leq n$ have that: $\tau(t_i)$ is the (estimated) execution time of t_i ; $s(t_i)$ is the set of statements of p covered by t_i ; $b(t_i)$ is the set of branches of p covered by t_i ; $m(t_i)$ is the set of MC/DC coverage items that are covered by t_i and $ms(t_i)$ is the value of mutation score that executed by t_i . Then, given $T' \subseteq T$, the five metrics used to drive optimization can be defined as Eq. (2) to (6).

- Total cost(execution time):

$$\tau(T') = \sum_{t_i \in T'} \tau(t_i) \quad (2)$$

- Total MS value

$$f(T') = \frac{|\cup_{t_i \in T'} ms(t_i)|}{|\cup_{t_i \in T'} ms(t_i)|} \quad (3)$$

- Total statement coverage:

$$s(T') = \frac{|\cup_{t_i \in T'} s(t_i)|}{|\cup_{t_i \in T} s(t_i)|} \quad (4)$$

- Total branch coverage:

$$b(T') = \frac{|\cup_{t_i \in T'} b(t_i)|}{|\cup_{t_i \in T} b(t_i)|} \quad (5)$$

- Total MC/DC coverage:

$$m(T') = \frac{|\cup_{t_i \in T'} m(t_i)|}{|\cup_{t_i \in T} m(t_i)|} \quad (6)$$

Naturally, we wish to minimize the first measure cost $\tau(T')$ and maximize the others, the mutation score $f(T')$ in particular.

B. Combination of Objectives

As mentioned before, there are totally five objectives used in our proposed approach, namely, mutant score, cost, statement coverage, branch coverage and MC/DC. The combination of objectives for cases with the participation of mutation score are shown in Table 3 ($\checkmark$ indicates the case include that objective).

TABLE III. OBJECTIVES COMBINATION FOR CASES WITH MUTATION SCORE

Objectives Number	Mutation score	Cost	Statement coverage	Branch coverage	MC/DC
3-obj	$\checkmark$	$\checkmark$	$\checkmark$		
4-obj	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	
5-obj	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

We can see from Table 3, the three-objectives(3-obj) case thus used mutation score, cost, and statement coverage. The four-objectives(4-obj) case added in the branch coverage of the test suite. Finally, the five-objectives(5-obj) case used all the 5 selected objectives, namely, mutation score, cost, statement coverage, branch coverage and MC/DC. The aim therefore was for these combinations of objectives to correspond to different classes of software of varying criticality, from ‘normal’ software (three objectives) to safety-critical software (five objectives).

C. Exploration Strategy

HV(Hypervolume) values could present valuable information about the relative effectiveness of the alternative optimization approaches. During the experimental exploration, for cases that with the participation of mutation score, we will try to focus on mutation score firstly and try to achieve the highest fault detection ability(*Mutation Score* = 1). On the basis of this, a lowest cost will be the second goal to achieve. And for cases that without the participation of the objective mutation score, achieve full coverage in each coverage criterion considered will be prioritized and then cost.

IV. EXPERIMENT DESIGN

A. Research Questions

The goal of our experiment is to explore the performance of the proposed mutation-based many-objective optimization approach in six different evolution algorithms. By our experiment, we try to answer the following research questions.

- **RQ1.** Does the mutation testing-based approach really improve the minimized test suite?

We split RQ1 into following 2 sub-questions to answer.

- **RQ1.1.** Does the cost of the test suite obtained by the case with the participation of mutation score lower than the case without mutation score for same evolution algorithm?

- **RQ1.2.** How are the fault detection ability of test suites obtained by the two different cases?

- **RQ2.** How does each many-objective optimization algorithm performs in each subject?

- **RQ3.** How does the proposed approach performs in large scale programs?

We address the above questions by conducting a number of experiments.

B. EMO algorithms applied

In this paper, we consider six many-objective optimization algorithms which were applied by H shown in Table 4.

TABLE IV. EXPERIMENTAL SUBJECTS

Algorithm Name	Description
NSGA-II	a classic Pareto-based algorithm ^[22]
IBEA	an indicator-based algorithm that uses a given performance indicator to guide the search ^[32]
MOEA/D-WS	a version of the decomposition-based algorithm MOEA/D, uses decomposition function weight sum ^[18]
MOEA/D-TCH	a version of the decomposition-based algorithm MOEA/D, uses decomposition function Tchebycheff ^[18]
MOEA/D-PBI	A versions of the decomposition-based algorithm MOEA/D, uses decomposition function penalty-based boundary intersection ^[18]
SPEA2+SDE	modifies the original SPEA by a shift-based density estimation in order to enable the algorithm to be suitable for many-objective problems ^[21]

These algorithms are representative in the evolution area and, apart from NSGA-II, all have been found to be effective in dealing with many-objective optimization problems with certain characteristics [21,29].

C. Subjects

The experiments used four subjects, with three obtained from the Software-artifact Infrastructure Repository (SIR) along with test suites. We used five smaller SIR subjects, they were: Schedule (a priority scheduler); tcas (an aircraft collision avoidance system); and tot_info (that takes input data and computes some statistics). We also used Space for the three objective problems since it is larger than the other SIR subjects used by Yoo and Harman, and obtained quite different results with Space (but very similar results with their other experimental subjects).

Table 5 provides details regarding the experimental subjects (name; lines of code; number of mutants, and test suite size).

TABLE V. EXPERIMENTAL SUBJECTS

Program	LOC (lines of code)	number of mutants	Test suite size
tcas	1608	71	1608
schedule	2650	41	2650
tot-info	1052	53	1052
space	6199	98	13585

Mutants were manually generated using some mutation operators. Those mutants are close to the correct version of the program, with the hope that they will be sufficient to simulate all faults. Incidentally, we note that the equivalent mutants problem is beyond the scope of this paper. In our study we rely on semi-manual detection of equivalent mutants.

D. Parameters setting

TABLE VI. PARAMETERS FOR ALL SIX ALGORITHMS

Objective Number	Population	Evaluation
3	100	80000
4	100	80000
5	100	80000

The size of the population in NSGA-II, MOEA/D-WS, MOEA/D-PBI, MOEA/D-TCH, NSGA2 and SPEA2+SDE was set to 100, and the Evaluation was set to 80000, as shown in Table 6.

V. RESULTS AND DISCUSSION

A. Answer to RQ1

To evaluate the effect of our presented mutation testing-based many-objective optimization approach, we performs a set of experiment on tcas(because tcas has the largest mutant amount in the subjects) with the objectives with and without the participation of mutation score separately, the result shows as Table 7. For RQ1.1, it is clear the cost of the minimized test suite obtained by the approach with mutation score under each algorithm is lower than that obtained by the approach without the participation of mutation score although they all reached the same statement coverage ratio. For RQ1.2, the two highlighted algorithms(MOEAD_PBI and NSGA2) each obtained two recommended result for the approach without the participation of mutation score, however, even for the better one, the killed mutant amount, which indicates the fault detection ability straightly, is much fewer than that in the case with the objective mutation score. As a result, we can reach an answer for RQ1, that is, the ability of the minimized test suite obtained by the many-objective optimization model with the participation of mutation testing is indeed better than that without it in both cost and fault detection ability.

TABLE VII. COMPARISON OF ABILITY FOR TESTSUITE OBTAINED BY CASE WITH AND WITHOUT MS(MUTATION SCORE)

	Cost		Statement coverage		Mutant killed	
	With MS	Without MS	With MS	Without MS	With MS	Without MS
IBEA	156433806	155675049	53	53	71	71
MOEAD_PBI	324769	81603722	53	53	71	71
		324641		53		34
MOEAD_TCH	6748760	60244618	53	53	71	71
MOEAD_WS	5809695	55115739	53	53	71	71
NSGA2	325587	324476	53	53	71	26
		215702		51		29
SPEA+SDE	57638173	159810136	53	53	71	71

TABLE VIII. NORMALIZED HV FOR SIR PROGRAMS WITH THREE OBJECTIVES.

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	0.180198	0.998968	0.963383	0.973733	1.000000	0.178389
tot-info	0.201014	0.997991	0.973234	0.981369	0.999984	0.198918
schedule	0.320396	0.998433	0.832326	0.886112	0.999412	0.311809

TABLE IX. NORMALIZED HV FOR SIR PROGRAMS AND FOUR OBJECTIVES.

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	0.138366	0.999032	0.712001	0.982946	1.000000	0.26311
tot-info	0.138963	0.998415	0.816236	0.998409	0.999988	0.138306
schedule	0.241152	0.99862	0.652095	0.989736	0.999995	0.237416

TABLE X. NORMALIZED HV FOR SIR PROGRAMS AND FIVE OBJECTIVES

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	0.120208	0.998952	0.935984	0.962149	0.999965	0.115347
tot-info	0.171396	0.997328	0.996585	0.997245	0.999927	0.126836
schedule	0.110158	0.998362	0.89308	0.970386	0.999948	0.107362

TABLE XI. COST OF THE OPTIMAL TEST SUITE FOR THREE OBJECTIVES

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	155453412	324713	6314894	4460580	324587	155018139
tot-info	343794012	914216	9421218	6028274	813962	344218927
schedule	423711841	658117	99206234	67951084	646142	423201986

TABLE XII. COST OF THE OPTIMAL TEST SUITE FOR FOUR OBJECTIVES

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	155018139	324689	52826884	2826139	433530	139448744
tot-info	370430142	819494	73531856	538881	866422	370844225
schedule	473340145	711504	206571269	5455943	501890	473918585

TABLE XIII. COST OF THE OPTIMAL TEST SUITE FOR FIVE OBJECTIVES

	IBEA	MOEA/D-PBI	MOEA/D-TCH	MOEA/D-WS	NSGA-II	SPEA2+SDE
tcas	139770145	324765	8812575	5550122	433530	140536597
tot-info	288552008	1131344	200835	1779172	1474764	353529021
schedule	472418486	887524	49691989	11984457	706041	474636551

B. Answer to RQ2

We use HV to establish the effectiveness of each algorithm. Table 8, 9, 10 show the HV results of SIR programs with three objectives, four objectives and five objectives respectively. The results clearly show that NSGA-II, MOEA/D-PBI, MOEA/D-TCH and MOEA/D-WS are significantly more effective than IBEA and SPEA2+SDE. Meanwhile, NSGA-II is the most effective one in all cases.

TABLE XIV. BEST-TEST SUITE ALGORITHMS

Objective number	Subjects	Best Test Suite	Subordinate
3 obj	tcas	NSGA-II	MOEA/D-PBI
	tot-info	NSGA-II	MOEA/D-PBI
	schedule	MOEA/D-PBI	NSGA-II
	space	MOEA/D-PBI	NSGA-II
4 obj	tcas	NSGA-II	MOEA/D-PBI
	tot-info	NSGA-II	MOEA/D-PBI, MOEA/D-WS
	schedule	NSGA-II	MOEA/D-PBI
5 obj	tcas	MOEA/D-PBI	NSGA-II
	tot-info	MOEA/D-PBI	NSGA-II, MOEA/D-TCH
	schedule	NSGA-II	MOEA/D-PBI

The purpose of test suite minimization is to reduce its execution cost. Table 11, 12 and 13 show the cost of the optimal test suite under each evolutionary algorithm for three objectives, four objectives and five objectives respectively. Table 14 gives the algorithm which gained the best test suite for the experiments across the 20 runs. For each model, the second column gives the best-performing algorithm, and the third column gives the other algorithms where the differences in cost values were closely. It is immediately clear that in all cases NSGA-II produces the best (cheapest) test suite for the six subject models. In contrast, MOEA/D-PBI produces the best (cheapest) test suite for the three subject models. Finally, as for space, MOEA/D-PBI produces the best (cheapest) test suite.

C. Answer to RQ3

To evaluate the performance of the proposed approach for large scale system, we use Space for the three objective problems since it is larger than the other SIR subjects and

applied in [11]. Table 15 shows the execute result of Space with HV value and Cost. From the result we can see MOEA/D-PBI performs the best with both the highest HV value 0.999295 and lowest Cost value 712267. Comparing with the worst result of SPEA2+SDE, MOEA/D-PBI only need 2.67%(712267/26686511) of its time cost, which means the proposed approach is applicable for large scale system and could effectively reduce the cost of test.

TABLE XV. EXPERIMENT RESULT OF SPACE WITH THREE OBJECTIVES

Algorithm	HV Value	Cost
IBEA	0.733022	26467703
MOEA/D-PBI	0.999295	712267
MOEA/D-TCH	0.912298	9546693
MOEA/D-WS	0.927719	7410605
NSGA-II	0.976187	2895755
SPEA2+SDE	0.732556	26686511

VI. CONCLUSION

This paper proposed a mutation testing-based many-objective optimization approach for regression test suite minimization. We adopted mutation testing result as an objective of optimization process as mutation score can be used to measure the effectiveness of a test suite in terms of its ability to detect faults. In the experiment, we applied six many-objective evolutionary algorithms: IBEA[32], NSGA-II[22], MOEA/D-PBI[28], MOEA/D-TCH[28], MOEA/D-WS[28] and SPEA2+SDE[21]. We used three smaller experimental subjects from the SIR repository plus a larger program Space. NSGA-II had the best performance in all the SIR programs with the differences being statistically significant in almost all cases. The relative performance of the other methods varied. Interestingly, for Space, MOEA/D-PBI performed better than NSGA-II. Overall, all the results show that the best test suites obtained by the proposed approach could significantly reducing the test cost for all subjects without reducing their fault detection ability. There are several lines of future work. There is a need to identify effective mutant generation way and perform additional experiments in which the methods are applied to a wider range of experimental subjects.

REFERENCES

- [1] M. Harman, Making the case for MORTO: Multi objective regression test optimization, in: *Proceedings of the ICST Workshops*, IEEE Computer Society, 2011, pp. 111–114.
- [2] J. Black, E. Melachrinoudis, D. Kaeli, Bi-criteria models for all-uses test suite reduction, in: *Proceedings of the 26th International Conference on Software Engineering (ICSE 2004)*, 2004, pp. 106–115.
- [3] T.Y. Chen, M.F. Lau, Dividing strategies for the optimization of a test suite, *Inf. Proc. Lett.* 60 (3) (1996) 135–141.
- [4] T.Y. Chen, M.F. Lau, A new heuristic for test suite reduction, *Inf. Softw. Technol.* 40 (5) (1998) 347–354.
- [5] M. Harman, Making the case for MORTO: Multi objective regression test optimization, in: *Proceedings of the ICST Workshops*, IEEE Computer Society, 2011, pp. 111–114.
- [6] Z. Li, M. Harman, R.M. Hierons, Search algorithms for regression test case prioritization, *IEEE Trans. Softw. Eng.* 33 (4) (2007) 225–237.
- [7] D. Jeffrey, N. Gupta, Improving fault detection capability by selectively retaining test cases during test suite reduction, *IEEE Trans. Softw. Eng.* 33 (2) (2007) 108–123.
- [8] S. Yoo, M. Harman, Regression testing minimization, selection and prioritization: a survey, *Softw. Test. Verif. Reliab.* 22 (2) (2012) 67–120.
- [9] Yoo S, Harman M. Using hybrid algorithm for Pareto efficient multi-objective test suite minimisation[J]. *Journal of Systems & Software*, 2010, 83(4):689-701.
- [10] S. Yoo, M. Harman, Pareto efficient multi-objective test case selection, in: *Proceedings of the ACM/SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2007)*, ACM, 2007, pp. 140–150.
- [11] Zheng W, Hierons R M, Li M, et al. Multi-objective optimisation for regression testing[J]. *Information Sciences*, 2016, s 334–335:1-16.
- [12] http://pitest.org/weak_tests/
- [13] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE computer*, 11(4):34–41, 1978.
- [14] P. J. Walsh. A measure of test completeness. PhD thesis, State University of New York at Binghamton, 1985.
- [15] P. G. Frankl, S. N. Weiss, and C. Hu. All-uses vs. mutation testing: An experimental comparison of effectiveness. *The Journal of Systems and Software*, 38(3):235–237, Sept. 1997.
- [16] J. Offutt, J. Pan, K. Tewary, and T. Zhang. An experimental evaluation of data flow and mutation testing. *Software—Practice and Experience*, 26(2):165–176, Feb.1996.
- [17] J. Offutt and R. H. Untch. Mutation 2000: Uniting the orthogonal. In *Mutation 2000: Mutation Testing in the Twentieth and the Twenty First Centuries*, San Jose, CA, pages 45–55, 2000.
- [18] Q. Zhang, H. Li, MOEA/D: A multiobjective evolutionary algorithm based on decomposition, *IEEE Trans. Evol. Comput.* 11 (6) (2007) 712–731.
- [19] Eckart Zitzler and Simon Kunzli. 2004. Indicator-based selection in multiobjective search. In "Proceedings of the International Conference on Parallel Problem Solving from Nature (PPSN'04). 832–842.
- [20] Hierons R M, Li M, Liu X, et al. SIP: Optimal Product Selection from Feature Models Using Many-Objective Evolutionary Optimization[J]. *Acm Transactions on Software Engineering & Methodology*, 2016, 25(2):17.
- [21] Miqing Li, Shengxiang Yang, and Xiaohui Liu. 2014b. A test problem for visual investigation of highdimensional multi-objective search. In *Proceedings of the IEEE Congress on Evolutionary Computation(CEC'14)*. 2140–2147.
- [22] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, *IEEE Trans. Evol. Comput.* 3 (4) (1999) 257–271.
- [23] Jia Y, Harman M. An Analysis and Survey of the Development of Mutation Testing[M]. IEEE Press, 2011.
- [24] J. Offutt, "The Coupling Effect: Fact or Fiction," *ACM SIGSOFT Software Engineering Notes*, vol. 14, no. 8, pp. 131–140, December 1989. [175] A. J.
- [25] Offutt, "Investigations of the Software Testing Coupling Effect," *ACM Transactions on Software Engineering and Methodology*, vol. 1, no. 1, pp. 5–20, January 1992.
- [26] J. Wegener, A. Baresel, H. Sthamer, Evolutionary test environment for automatic structural testing, *Inf. Softw. Technol.* 43 (14) (2001) 841–854.
- [27] Roberto E. Lopez-Herrejon, Lukas Linsbauer, and Alexander Egyed. 2015. A systematic mapping study of search-based software engineering for software product lines. *Information and Software Technology* 61 (2015), 3351.DOI:<http://dx.doi.org/10.1016/j.infsof.2015.01.008>.
- [28] Ali S, Briand L C, Hemmati H, et al. A Systematic Review of the Application and Empirical Investigation of Search-Based Test Case Generation[J]. *IEEE Transactions on Software Engineering*, 2009, 36(6):742-762.
- [29] Tobias Wagner, Nicola Beume, and Boris Naujoks. 2007. Pareto-, aggregation-, and indicator-based methods in many-objective optimization. In *Proceedings of the 4th International Conference on Evolutionary Multi-Criterion Optimization (EMO'07)*. 742–756.
- [30] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*, John Wiley, New York, 2001.
- [31] Tobias Wagner, Nicola Beume, and Boris Naujoks. 2007. Pareto-, aggregation-, and indicator-based methods in many-objective optimization. In *Proceedings of the 4th International Conference on Evolutionary Multi-Criterion Optimization (EMO'07)*. 742–756.
- [32] D.A.V. Veldhuizen, G.B. Lamont, Evolutionary computation and convergence to a Pareto front, *Late Break. Papers Genet. Program. Conf.* (1998) 221–228.
- [33] M. Li, S. Yang, X. Liu, Bi-goal evolution for many-objective optimization problems, *Arti. Intell.* 228 (2015) 45–65.
- [34] H. Ishibuchi, Y. Hitotsuyanagi, N. Tsukamoto, Y. Nojima, Many-objective test problems to visually examine the behavior of multiobjective evolution in adecision space, in: *Proceedings of the Parallel Problem Solving from Nature*, 2010, pp. 91–100.
- [35] S. Holm, A simple sequentially rejective multiple test procedure, *Scand. J. Stat.* 6 (2) (1979) 65–70.